

ОТЧЕТ

о выполнении задания по курсу «Суперкомпьютерное моделирование и технологии»

Вариант: 4

Выполнил: Назаров Михаил Эрнестович

Группа: 621

1. Математическая постановка задачи (Вариант 4)

В области $\Omega = [0, L_x] \times [0, L_y] \times [0, L_z]$ решается уравнение:

$$\frac{\partial^2 u}{\partial t^2} = a^2 \Delta u$$

Граничные условия (Вариант 4):

- По оси X: Граничные условия 1-го рода:
 $u(0, y, z, t) = 0, \quad u(L_x, y, z, t) = 0$
- По оси Y: Периодические граничные условия:
 $u(0, y, z, t) = 0, \quad u(L_x, y, z, t) = 0$
- По оси Z: Периодические граничные условия:
 $u(x, y, 0, t) = u(x, y, L_z, t)$

Начальные условия:

$$u|_{\{t=0\}} = \varphi(x, y, z)$$

$$\frac{\partial u}{\partial t}|_{t=0} = 0$$

Аналитическое решение:

$$u_{an} = \sin\left(\frac{3\pi x}{L_x}\right) \sin\left(\frac{2\pi y}{L_y}\right) \sin\left(\frac{2\pi z}{L_z}\right) \cos(a_t t + 4\pi)$$

2. Численный метод и схема распараллеливания

2.1 Декомпозиция области (MPI)

Используется трехмерная декомпозиция области (блочное разбиение).

Каждый MPI-процесс отвечает за свой подблок сетки размером

$$N_{loc_x} \times N_{loc_y} \times N_{loc_z}.$$

Для обмена данными между соседними блоками используется слой теневых ячеек (halo cells) толщиной в 1 узел.

- **Ось X:** Обмен нециклический.
- **Оси Y, Z:** Обмен циклический (с использованием MPI_Sendrecv и топологии MPI_Cart_create с параметром periods={0, 1, 1}).

2.2 Реализация на GPU (CUDA)

Каждый MPI-процесс управляет одним GPU (принцип "1 процесс — 1 GPU").

Основные этапы на GPU:

1. **Инициализация:** Ядро init_kernel задает начальные условия непосредственно в памяти видеокарты.
2. **Вычисления:** Ядра step_kernel обновляют значения во внутренних точках подблока.
3. **Обмены (Halo Exchange):**
 - Данные границ упаковываются в буферы (ядро pack_kernel).
 - Копируются на хост (cudaMemcpy DeviceToHost).
 - Пересылаются соседям через MPI_Sendrecv.
 - Копируются обратно на устройство (cudaMemcpy HostToDevice).

- Распаковываются в теневые слои (ядро unpack_kernel).
4. **Граничные условия:** Ядро boundary_kernel явно зануляет границы по оси X.

3. Результаты экспериментов

3.1 Проверка корректности

Корректность проверялась путем сравнения численного решения с аналитическим на каждом шаге по времени.

Максимальная ошибка на сетке 128^3 после 50 итераций составила: 0.00013794, а на сетке 256^3 : 0.00002038, что соответствует порядку аппроксимации схемы $O(h^2 + \tau^2)$.

3.2 Детализация времени выполнения (MPI + CUDA)

Замеры для сетки 128^3 на 50 шагах по времени.

Кол-во GPU	Инициализация (T_init)	Вычисления (T_calc)	Копирование H2D/D2H (T_copy)	Коммуникации MPI (T_comm)	Общее время (T_total)
1	0.00028266с	0.21538551с	0.04767606с	0.01319863с	0.27654288с
2	0.00014547с	0.13199190с	0.04256615с	0.01525027с	0.18995378с
4	0.00006758с	0.07628524с	0.01455670с	0.05313832с	0.14404785с
8	0.00003280с	0.11644135с	0.04572459с	0.20956471с	0.37176349с

Замеры для сетки 256^3 на 50 шагах по времени.

Кол-во GPU	Инициализация (T_init)	Вычисления (T_calc)	Копирование H2D/D2H (T_copy)	Коммуникации MPI (T_comm)	Общее время (T_total)
1	0.00128298с	1.12041553с	0.03507549с	0.01012877с	1.16690271с
2	0.00124458с	0.58726080с	0.01959865с	0.41646280с	1.02456677с
4	0.00051859с	0.36312857с	0.02261083с	0.26846701с	0.65472498с
8	0.00022922с	0.49549283с	0.01800276с	0.39592499с	0.90964978с

Замеры для сетки 512^3 на 50 шагах по времени.

Кол-во GPU	Инициализация (T_init)	Вычисления (T_calc)	Копирование H2D/D2H (T_copy)	Коммуникации MPI (T_comm)	Общее время (T_total)
1	0.02437056с	1.79174316с	0.03793538с	0.02238451с	1.87643359с
2	0.01205274с	0.87720416с	0.03245101с	0.12603485с	1.04774268с

Примечание:

- T_calc включает время работы ядер CUDA (расчет + упаковка/распаковка).
- T_copy — время пересылки буферов между хостом и GPU.

- T_{comm} — время работы MPI_Sendrecv и MPI_Reduce.

3.3 Сравнение производительности версий

Ускорение вычислено по следующей формуле: $S = \frac{T_{serial}}{T_{parallel}}$

Где T_{serial} – время последовательной программы,

$T_{parallel}$ – время параллельной программы

Эффективность вычислена по следующей формуле: $E = \frac{S}{p} * 100\%$

Где S – ускорение, p – количество использованных процессов.

На графиках эффективность представлена как число до умножения на 100%.

Сравнение времени работы различных реализаций задачи на сетке 128^3 .

Версия программы	Кол-во процессов/нитей	Время (сек)	Ускорение (S)	Эффективность (E)
Последовательная	1	0.61613348	1.0	100%
OpenMP	1	1.38943904	0.4434404549	44.34404549%
OpenMP	2	0.69839722	0.8822106709	44.11053355%
OpenMP	4	0.46584010	1.322628687	33.06571717%
OpenMP	8	0.29288792	2.103649341	26.29561676%

MPI	1	0.56848845	1.083810023	108.3810023%
MPI	2	0.36053554	1.708939651	85.44698255%
MPI	4	0.23750822	2.594156446	64.85391115%
MPI	8	0.19540155	3.153165776	39.4145722%
MPI + CUDA	1	0.27654288	2.227985331	222.7985331%
MPI + CUDA	2	0.18995378	3.243596837	162.1798419%
MPI + CUDA	4	0.14404785	4.277283417	106.9320854%
MPI + CUDA	8	0.37176349	1.65732649	20.71658112%

Сравнение времени работы различных реализаций задачи на сетке 256^3 .

Версия программы	Кол-во процессов/нитей	Время (сек)	Ускорение (S)	Эффективность (E)
Последовательная	1	3.49928793	1.0	100%
OpenMP	2	5.71990886	0.6117733719	30.5886686%

OpenMP	4	3.11613420	1.122958032	28.0739508%
OpenMP	8	2.66580819	1.312655555	16.40819444%
MPI	1	4.29305308	0.8151047436	81.51047436%
MPI	4	1.16653292	2.999733544	74.9933386%
MPI	8	0.93079478	3.759462349	46.99327936%
MPI + CUDA	1	1.16690271	2.998782932	299.8782932%
MPI + CUDA	2	1.02456677	3.415383001	170.76915%
MPI + CUDA	4	0.65472498	5.344668429	133.6167107%
MPI + CUDA	8	0.90964978	3.84685184	48.085648%

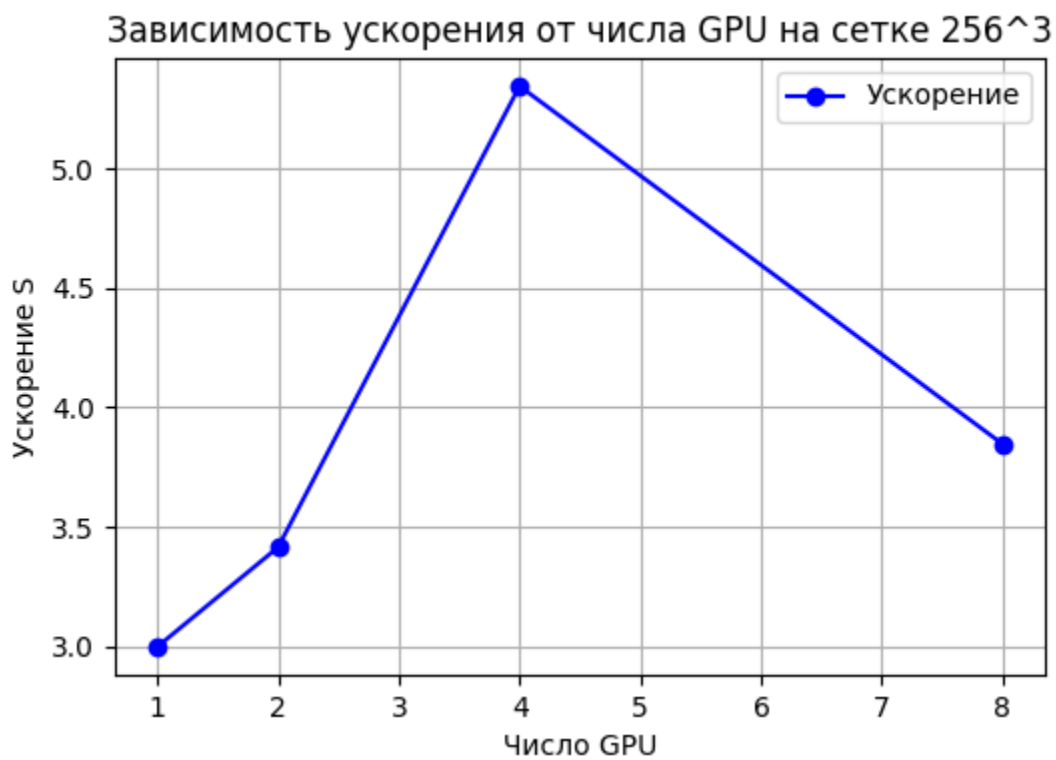
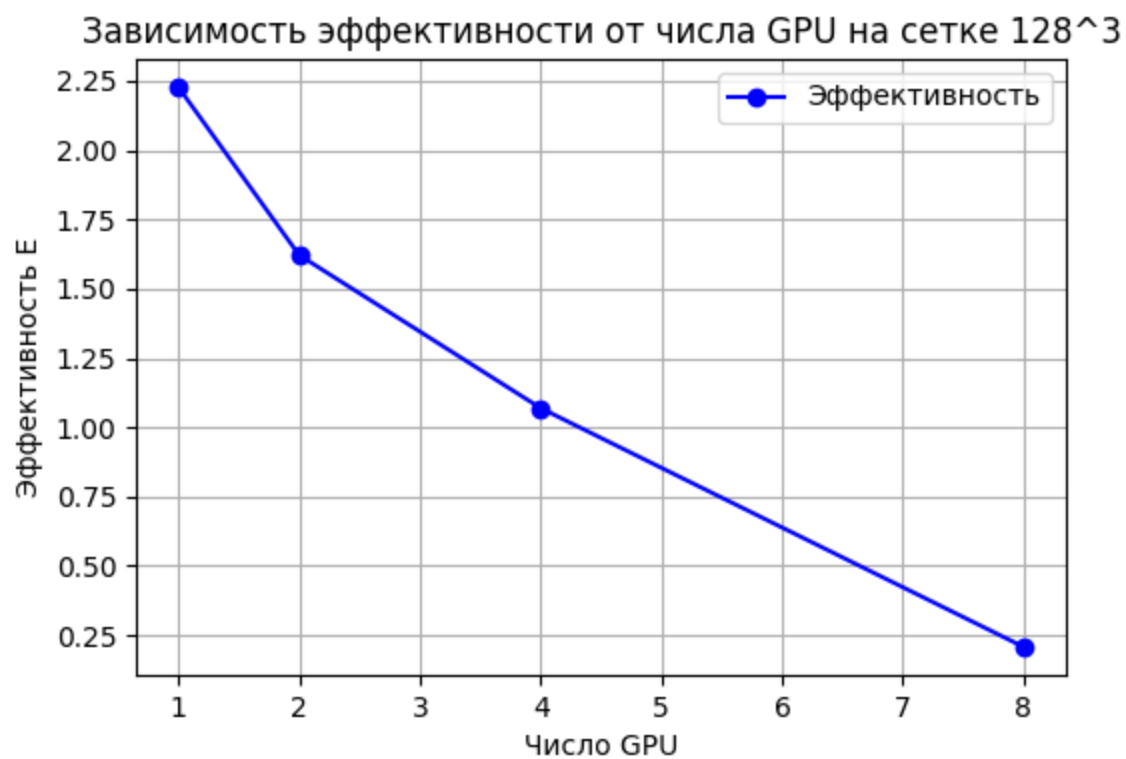
Сравнение времени работы различных реализаций задачи на сетке 512^3

Версия программы	Кол-во процессов/нитей	Время (сек)	Ускорение (S)	Эффективность (E)
Последовательная	1	28.12963195	1.0	100%

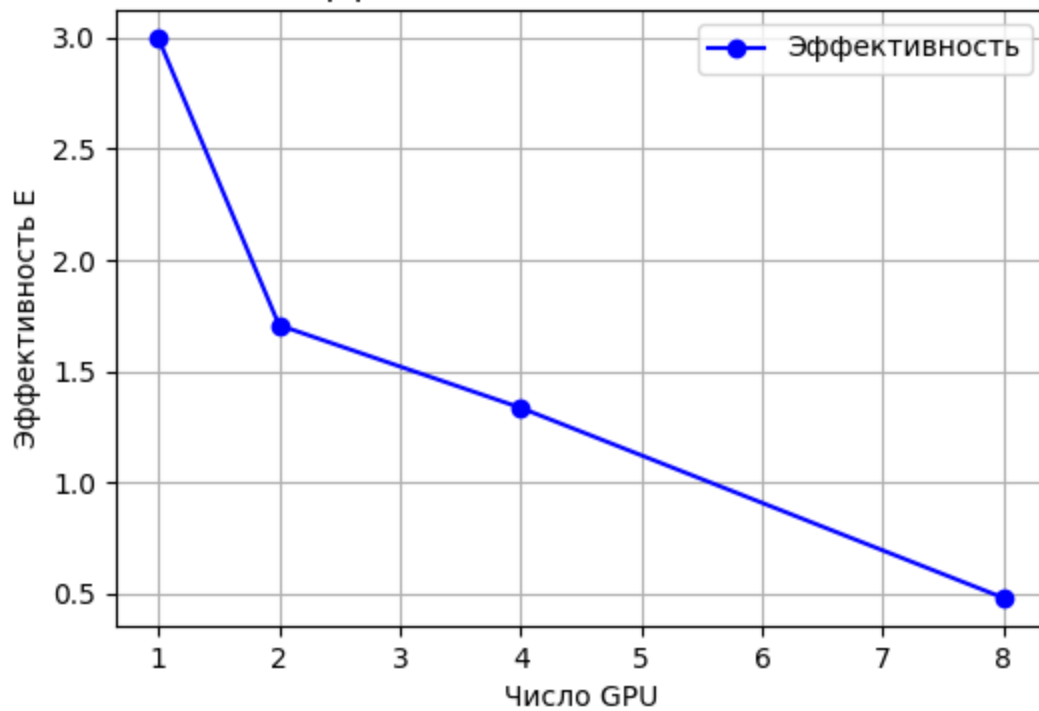
MPI	20	4.40737437	6.382401309	31.91200654%
OpenMP + MPI	20/8	4.61969504	6.089066855	30.44533428%
MPI + CUDA	1	1.87643359	14.99100853	1499.100853%
MPI + CUDA	2	1.04774268	26.84784393	1342.392196%

3.4 Графики

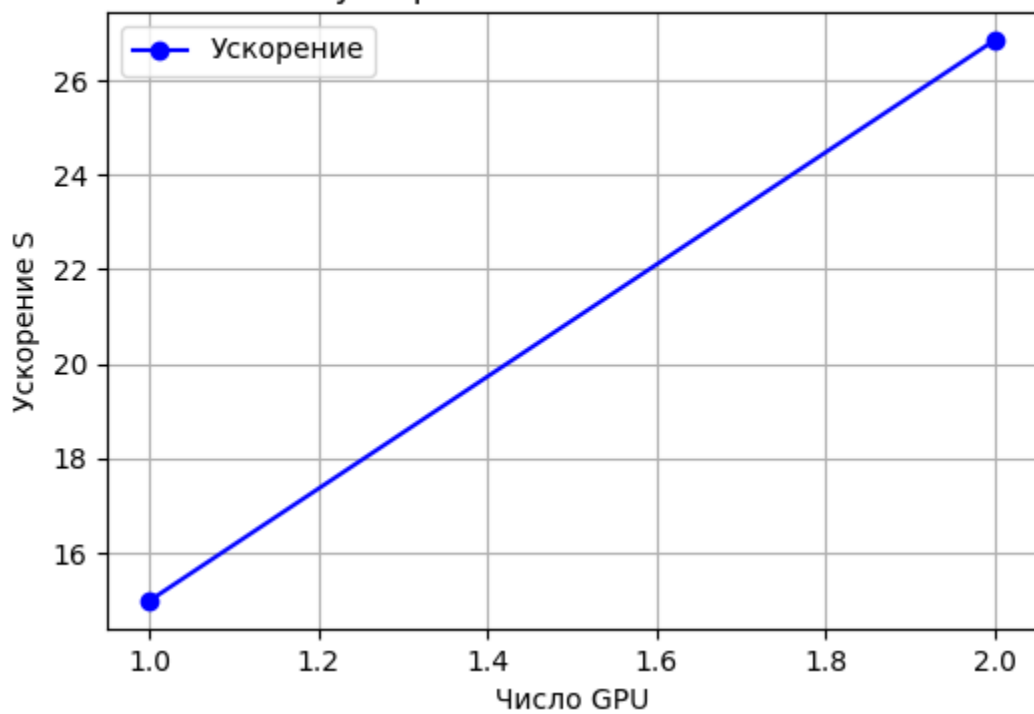


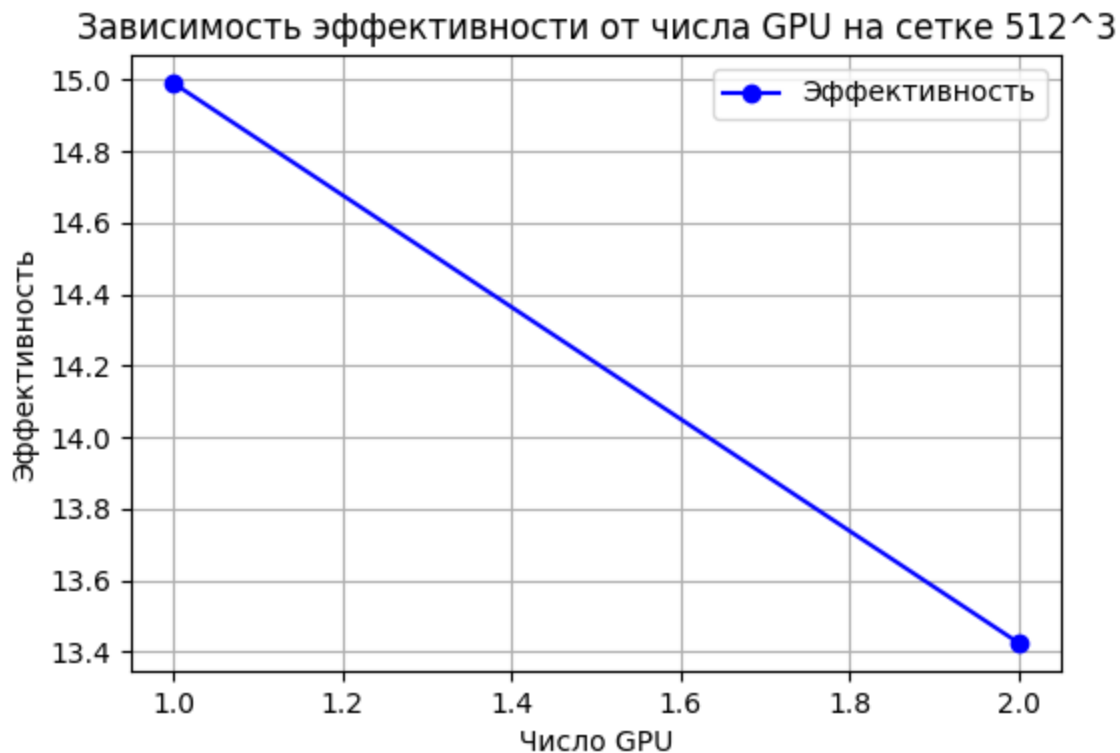


Зависимость эффективности от числа GPU на сетке 256^3



Зависимость ускорения от числа GPU на сетке 512^3





4. Анализ результатов

1. Версия MPI+CUDA показала ускорение по сравнению с последовательной, OpenMP и MPI версиями на CPU. Это объясняется высокой параллельной и вычислительной способностью GPU Tesla P100.
2. Как видно из таблиц 3.2, существенную часть времени занимают вычисления T_{calc} и MPI-коммуникации T_{comm} .
3. В целом для всех использованных методов параллелизации вычислений падает эффективность с увеличением количества процессов/нитей.
4. При использовании 8 GPU время работы программы повышается из-за чрезмерного разбиения сетки и ограничений в скорости коммуникации между процессами. Также данные всех запусков подвержены некоторым случайным отклонениям, которые зависят от загруженности суперкомпьютера Polus.

5. Абсолютные значения эффективности и ускорения выше на сетке 256^3 , несмотря на общее сохранение формы графика. Это, вероятно, связано с большей корректностью использования GPU вычислений с большим размером сетки.

5. Ссылка на репозиторий

<https://github.com/PayWga/SuperComp2025/tree/CUDA>